Experiments:

Stage	Learning Signal / Inputs	What We Updated	Resulting Behavior	Links
1. Initial (loss-only learning)	Primarily training loss and immediate relative improvement	Core action policy (keep, LR up/down, densify/prune adjustments) trained from step-to-step loss-based reward lr_up_3pct / lr_down_3pct 100% step-loss, 0% trend cooldown = 2, gate_alpha = 0.6	It converged earlier to the final loss, but densification was too aggressive, which caused too slow training. But in the end the final loss was same.	
2. Expanded learning inputs (eval-time quality context)	Added broader quality context from eval-side signals (SSIM/LPIPS/PSNR-type monitoring context) alongside training progress signals, phase-based learning (initial, densification start, end).	Integrated richer telemetry/comparison context into the decision ecosystem and monitoring logic, not only raw loss trend. lr_up_3pct / lr_down_3pct 65% step-loss, 35% trend cooldown = 2, gate_alpha = 0.6, small_change_band = 0.015, quality_priority_gate_multiplier = 2.0	More informed than pure loss, but now it didn't make any changes to the loss curve. So need to harden the rule.	
3. (safeguard + trend + tuning updates)	Blended reward with trend signal plus safety gating/cooldown and late-phase quality protection	Added adaptive gate thresholding, cooldown, stable-small-change keep behaviour, late-phase quality priority, run/phase trend scope, and latest update: lower trend weight (10%), LR step to 5%, prune-down action	Much softer behaviour; often tracks baseline loss curve shape closely, with many keep decisions and limited net improvement	base vs lr3pct —---------- Base vs lr5pct Log Ai log

4. Keep loss effect 70%	Rebalance Keep the new safety/action set, but make loss dominate again	lr_up_5pct / lr_down_5pct 70% step-loss, 30% trend same as stage 3,		

Tried with multiple values for different parameters:

#	Run Name	LR↑	LR↓	Gate	CD	Small	Reward	Best Loss	Step	Duration
1	mx12_01_u10_d15_g06_c0_s0002_r90	+10%	-15%	0.06	0	0.002	90%	0.0901	4200	32m 59s
2	mx12_02_u20_d30_g01_c0_s0002_r80	+20%	-30%	0.01	0	0.002	80%	0.1015	4200	3m 54s
3	mx12_03_u30_d10_g03_c1_s0002_r70	+30%	-10%	0.03	1	0.002	70%	0.0864	4200	4m 09s
4	mx12_04_u10_d15_g06_c1_s0002_r80	+10%	-15%	0.06	1	0.002	80%	0.0818	4200	63m 28s
5	mx12_05_u20_d30_g01_c2_s0006_r70	+20%	-30%	0.01	2	0.006	70%	0.0813	4200	3m 47s
6	mx12_06_u30_d10_g03_c2_s0006_r90	+30%	-10%	0.03	2	0.006	90%	0.0783	4200	4m 04s
7	mx12_07_u10_d15_g06_c0_s0006_r70	+10%	-15%	0.06	0	0.006	70%	0.0817	4200	4m 09s

8	mx12_08_u20_d30_g01_c0_s0006_r90	+20%	-30%	0.01	0	0.006	90%	0.0814	4200	4m 18s
9	mx12_09_u30_d10_g03_c1_s0015_r80	+30%	-10%	0.03	1	0.015	80%	0.0800	4200	4m 04s
10	mx12_10_u10_d15_g06_c1_s0015_r90	+10%	-15%	0.06	1	0.015	90%	0.0811	4200	23m 54s
11	mx12_11_u20_d30_g01_c2_s0015_r80	+20%	-30%	0.01	2	0.015	80%	0.0813	4200	23m 44s
12	mx12_12_u30_d10_g03_c2_s0015_r70	+30%	-10%	0.03	2	0.015	70%	0.0777 (best)	4200	3m 52s

Annex 2 contains the parameters used for each.

1) X parameters (input features)

Each decision step builds:

```
None
X ∈ ℝ²⁰
```

So every controller decision is based on a 20-dimensional feature vector.

sn	Feature name	Meaning
1	loss	Current training loss
2	relative_improvement	Improvement relative to previous loss
3	slope	Trend slope over recent loss window
4	variance	Variance over recent loss window
5	recent_delta	Latest local loss-change signal
6	gaussians	Current Gaussian count
7	gaussian_growth	Growth rate of Gaussian count
8	lr_means	Learning rate for means optimizer group
9	lr_opacities	Learning rate for opacities group
10	lr_sh0	Learning rate for SH0 group
11	lr_shN	Learning rate for SHN group
12	grow_grad2d	Current densify threshold
13	prune_opa	Current prune threshold
14	refine_every	Strategy refine interval
15	reset_every	Strategy reset interval
16	progress	Training progress from 0 to 1
17	phase_one_hot_0	Early-phase flag

18	phase_one_hot_1	Mid-phase flag
19	phase_one_hot_2	Late-phase flag
20	phase	Numeric phase identifier

The relative improvement feature is:

$$\text{relative_improvement} = \frac{\text{prev_loss} - \text{loss}}{|\text{prev_loss}|}$$

Example feature vector:

```
None
X = [
  0.0231,
  0.0042,
  -0.0003,
  1.2e-5,
  -0.0001,
  154321,
  0.018,
  0.00016,
  0.0009,
  0.0025,
  0.00025,
  0.0002,
  0.004,
  100,
  3000,
  0.62,
  0,
  1,
  0,
  1
]
```

Interpretation:

- Current loss = 0.0231
- Improvement from previous step = 0.42%
- Training is in the mid phase (phase_one_hot_1 = 1)
- Progress is 62%
- The controller currently sees ~154k Gaussians and current LR / densify / prune settings

2) Y parameter (training target)

The controller learns from the reward observed after taking an action.

The target reward is:

$$y = \text{clip}\left(w_s r_{\text{step}} + w_t r_{\text{trend}},\ -0.05,\ 0.05\right)$$

Where:

- r_step = immediate step-to-step improvement reward
- r_trend = longer-term trend reward
- w_s and w_t = normalized reward weights from:
    - ai_reward_step_weight
    - ai_reward_trend_weight

The step reward is:

$$r_{\text{step}} = \text{clip}\left(\frac{\text{prev_loss} - \text{loss}}{|\text{prev_loss}|},\ -0.05,\ 0.05\right)$$

Example settings:

```
None
ai_reward_step_weight  = 0.8
ai_reward_trend_weight = 0.2
r_step  = 0.012
```

```
r_trend = 0.004
```

Then the target becomes:

y = 0.8(0.012)+0.2(0.004)=0.0104

So the controller is told that the action taken was worth approximately +0.0104 reward.

3) ŷ (model prediction)

The MLP predicts one value for every possible action:

```
None
ŷ = [ ŷ_keep, ŷ_lr_up, ŷ_lr_down, ŷ_densify_up, ŷ_densify_down, ŷ_prune_up,
ŷ_prune_down ]
```

The controller chooses the action with the highest predicted value:

$$a = \arg\max_i \hat{y}_i$$

Only the selected action is trained against the observed reward.
So if the chosen action is a, then:

$$\hat{y} = \hat{y}_a$$

and the update tries to move that prediction toward the true target reward:

$$(\hat{y}_a - y)^2$$

Example:

```
None
Predicted scores:
[0.002, 0.011, 0.006, -0.001, 0.003, 0.000, -0.002]
```

This corresponds to:

Action	Predicted score

keep	0.002
lr_up	0.011
lr_down	0.006
densify_up	-0.001
densify_down	0.003
prune_up	0.000
prune_down	-0.002

The highest score is for lr_up, so the controller chooses lr_up.

Suppose the actual observed reward after the next step is:

```
None
y = 0.0104
```

Then the controller updates only the lr_up output, pushing:

```
None
0.011 → 0.0104
```

while leaving the other action outputs unchanged.

Annex 1:

Overall controller logic for LR, densify, and prune actions

An LR, densify, or prune change only happens if:

1. the current step is inside the valid tuning window,
2. the specific LR or strategy action is enabled,
3. the controller is not blocked by cooldown or missing history,
4. the recent improvement is not already good enough,
5. the recent change is not too small,
6. late-phase quality protection allows the action,
7. and only then are LR, densify, or prune values actually changed.

1. Common preconditions in the training loop

A tuning decision is considered only when all of the following are true:

- The run is in modified mode.
- The current step is inside the effective tuning horizon:
  - not past the effective end,
  - not before the earliest start,
  - matches the tune interval cadence (unless it is an exact end step),
  - and has not already been tuned on the same step.
- At least one control window is open:
  - LR window open:

```
None
- tune_start <= step <= tune_end
```

- Strategy window open:

```
None
- strategy_start <= step <= strategy_end
```

- If both windows are closed, then no action is taken.

2. Action-specific window gates

Even if the step is valid, the specific action type must also be enabled:

- LR actions are allowed only if apply_lr == True
- Densify and prune actions are allowed only if apply_strategy == True
- Strategy actions can also be blocked if strategy_frozen == True

3. Controller-level keep-forcing conditions

Even when the step is eligible and the action type is enabled, the controller may still override everything and force a keep action.

This happens when:

- Cooldown is active
  → selected action becomes keep
- No previous loss exists yet
  → controller cannot measure improvement, so it uses keep
- The current improvement is already large enough
  → controller decides no intervention is needed

The relative improvement is:

$$\text{rel_improve}=\frac{\text{previous_loss}-\text{current_loss}}{\text{previous_loss}}$$

Example:

```
None
```

- previous_loss = 0.100
- current_loss = 0.0995

Then:

```
None
```

- rel_improve = (0.100 - 0.0995) / 0.100 = 0.005

which means a 0.5% improvement.

The controller compares this against a required threshold called gate_threshold.

4. How gate_threshold is computed

The required improvement threshold is:

$$\text{gate_threshold}=\mathrm{clamp}\left(\text{phase_base}+\alpha\sigma,;0.0005,;0.08\right)$$

Where:

- phase_base = baseline improvement target for the current phase of training
- α (gate_alpha) = how much recent noise should matter
- σ = recent loss volatility
- clamp(..., 0.0005, 0.08) = keep the threshold between 0.05% and 8%

Phase base

With the settings we have:

- base_min_improvement = 0.005
- progress = 0.40

the phase base is approximately:

$$\text{phase_base}=0.005 + 0.002\cdot0.40 = 0.0058$$

So before considering noise, the controller already wants about a 0.58% improvement.

Phase:
The phase changes the controller in a few specific ways:

It switches the trend reward scope if trend_scope="phase" so each phase has its own slope/history.
It changes the gate target slightly:

- phase 0: base_min_improvement + 0.004
- phase 1: base_min_improvement
- phase 2: base_min_improvement - 0.002

Base improvement: How much percent better does the current loss need to be versus the previous loss before the controller considers it good enough to keep
It also feeds a one-hot phase into the model features, so the policy can learn different behaviour per phase.

Recent loss volatility

Recent loss volatility means how much the recent loss values fluctuate. It is measured as:

$$\sigma=\sqrt{\mathrm{variance}(\text{recent loss window})}$$

- Small σ = smooth/stable losses
- Large σ = noisy losses

Suppose:

```
None
```

- sigma = 0.003
- alpha = 0.6

Then the volatility penalty is:

$$\alpha\sigma = 0.6\cdot0.003 = 0.0018$$

So the final threshold becomes:

$$\text{gate_threshold}=0.0058+0.0018=0.0076$$

Interpretation:

- If rel_improve >= 0.0076 (0.76%), the controller keeps the current settings.

- If rel_improve < 0.0076, the controller becomes more willing to change LR, densify, or prune.

So:

- phase_base = normal required improvement
- σ = measured recent noise
- ασ = extra penalty because of noise
- gate_threshold = final minimum improvement required before the controller says "keep"

5. Small-change keep rule

Even if the improvement is slightly negative, the controller may still force keep if the change is considered too small to matter.

This uses small_change_band.

For the current settings:

```
None
- small_change_band = 0.015
```

If the relative improvement is still above:

```
None
- -0.015
```

then the controller treats the change as minor and still keeps the current settings.

Examples:

```
None
- rel_improve = -0.004   -> keep
- rel_improve = -0.010   -> keep
```

```
- rel_improve = -0.020   -> action allowed
```

because only losses worse than -1.5% are considered large enough to justify intervention.

6. Late quality-priority gate

In the later phase of training there is an additional protection layer.

After:

```
None
- quality_priority_start_step = 70% of max_steps
```

certain “risky” actions require stricter conditions before they are allowed.

Risky actions include:

```
None
- lr_up
- densify_up
- densify_down
- prune_up
```

If these actions do not pass the stricter late-phase rule, they are replaced with keep.

The purpose is to avoid damaging image quality late in training when the model is already close to convergence.

7. What actually changes when an action is finally allowed

Only after all previous checks pass does the controller actually modify parameters.

- LR up/down changes the learning rate by the configured LR amount.
- Densify up/down changes grow_grad2d by ±2%.
- Prune up/down changes prune_opa by +1% / -1%.

Current configured magnitudes:

```
None
```

- LR up = +5%
- LR down = -5%
- grow_grad2d change = ±2%
- prune_opa change = ±1%

Annex 2: Merged Run Summary

Run Name	Parameters	PDF File
mx12_01_u10_d15_g06_c0_s0002_r90	ai_lr_up_multiplier=1.10; ai_lr_down_multiplier=0.85; ai_gate_alpha=0.60; ai_cooldown_intervals=0; ai_small_change_band=0.002; ai_reward_step_weight=0.90; ai_reward_trend_weight=0.10	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_01_u10_d15_g06_c0_s0002_r90_telemetry_20260413_175920.pdf
mx12_02_u20_d30_g01_c0_s0002_r80	ai_lr_up_multiplier=1.20; ai_lr_down_multiplier=0.70; ai_gate_alpha=0.10; ai_cooldown_intervals=0; ai_small_change_band=0.002; ai_reward_step_weight=0.80; ai_rewar20	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_02_u20_d30_g01_c0_s0002_r80_telemetry_20260413_175920.pdf
mx12_03_u30_d10_g03_c1_s0002_r70	ai_lr_up_multiplier=1.30; ai_lr_down_multiplier=0.90; ai_gate_alpha=0.30; ai_cooldown_intervals=1; ai_small_change_band=0.002; ai_reward_step_weight=0.70; ai_reward_trend_weight=0.30	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_03_u30_d10_g03_c1_s0002_r70_telemetry_20260413_175920.pdf
mx12_04_u10_d15_g06_c1_s0002_r80	ai_lr_up_multiplier=1.10; ai_lr_down_multiplier=0.85; ai_gate_alpha=0.60; ai_cooldown_intervals=1; ai_small_change_band=0.002; ai_reward_step_weight=0.80; ai_reward_trend_weight=0.20	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_04_u10_d15_g06_c1_s0002_r80_telemetry_20260413_175920.pdf
mx12_05_u20_d30_g01_c2_s0006_r70	ai_lr_up_multiplier=1.20; ai_lr_down_multiplier=0.70; ai_gate_alpha=0.10; ai_cooldown_intervals=2; ai_small_change_band=0.006; ai_reward_step_weight=0.70; ai_reward_trend_weight=0.30	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_05_u20_d30_g01_c2_s0006_r70_telemetry_20260413_175920.pdf
mx12_06_u30_d10_g03_c2_s0006_r90	ai_lr_up_multiplier=1.30; ai_lr_down_multiplier=0.90; ai_gate_alpha=0.30; ai_cooldown_intervals=2; ai_small_change_band=0.006; ai_reward_step_weight=0.90; ai_reward_trend_weight=0.10	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_06_u30_d10_g03_c2_s0006_r90_telemetry_20260413_175920.pdf

mx12_07_u10_d15_g06_c0_s0006_r70	ai_lr_up_multiplier=1.10; ai_lr_down_multiplier=0.85; ai_gate_alpha=0.60; ai_cooldown_intervals=0; ai_small_change_band=0.006; ai_reward_step_weight=0.70; ai_reward_trend_weight=0.30	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_07_u10_d15_g06_c0_s0006_r70_telemetry_20260413_175921.pdf
mx12_08_u20_d30_g01_c0_s0006_r90	ai_lr_up_multiplier=1.20; ai_lr_down_multiplier=0.70; ai_gate_alpha=0.10; ai_cooldown_intervals=0; ai_small_change_band=0.006; ai_reward_step_weight=0.90; ai_reward_trend_weight=0.10	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_08_u20_d30_g01_c0_s0006_r90_telemetry_20260413_175921.pdf
mx12_09_u30_d10_g03_c1_s0015_r80	ai_lr_up_multiplier=1.30; ai_lr_down_multiplier=0.90; ai_gate_alpha=0.30; ai_cooldown_intervals=1; ai_small_change_band=0.015; ai_reward_step_weight=0.80; ai_reward_trend_weight=0.20	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_09_u30_d10_g03_c1_s0015_r80_telemetry_20260413_175921.pdf
mx12_10_u10_d15_g06_c1_s0015_r90	ai_lr_up_multiplier=1.10; ai_lr_down_multiplier=0.85; ai_gate_alpha=0.60; ai_cooldown_intervals=1; ai_small_change_band=0.015; ai_reward_step_weight=0.90; ai_reward_trend_weight=0.10	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_10_u10_d15_g06_c1_s0015_r90_telemetry_20260413_175921.pdf
mx12_11_u20_d30_g01_c2_s0015_r80	ai_lr_up_multiplier=1.20; ai_lr_down_multiplier=0.70; ai_gate_alpha=0.10; ai_cooldown_intervals=2; ai_small_change_band=0.015; ai_reward_step_weight=0.80; ai_reward_trend_weight=0.20	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_11_u20_d30_g01_c2_s0015_r80_telemetry_20260413_175921.pdf
mx12_12_u30_d10_g03_c2_s0015_r70	ai_lr_up_multiplier=1.30; ai_lr_down_multiplier=0.90; ai_gate_alpha=0.30; ai_cooldown_intervals=2; ai_small_change_band=0.015; ai_reward_step_weight=0.70; ai_reward_trend_weight=0.30	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx12_12_u30_d10_g03_c2_s0015_r70_telemetry_20260413_175921.pdf
mx13_01_extreme_up	—	7a10c332-ce34-4b0f-bc79-da8c7385d97f_mx13_01_extreme_up_telemetry_20260413_175921.pdf

Folder to pdf:
https://drive.google.com/drive/folders/1BWj5h2_nvD1aTxKP2hshKm9_ONulN_Ct?usp=drive_link